Testing Plan Document

University Internship & Industry Matching Portal

---

1. Introduction

1.1 Purpose

This document provides a detailed and comprehensive testing strategy for the University Internship & Industry Matching Portal. It outlines testing types, methodologies, test cases, and responsibilities to ensure the system meets functional and non-functional requirements.

1.2 Scope

The testing plan covers:

- Functional testing of all modules (Student, Company, Admin)
- Integration testing of frontend and backend
- System testing on Azure deployment
- Performance, security, and usability testing
- Database testing
- Regression testing for updates

---

2. Objectives

- Ensure the system meets SRS specifications
- Detect and fix defects early in development
- Verify system functionality, performance, and security
- Confirm data integrity and reliability
- Provide a robust, stable, and user-friendly application

---

3. Testing Methodology

- Agile-based testing integrated with development sprints
- Unit testing for individual components
- Integration testing for API and frontend communication
- System testing on deployed Azure environment
- Regression testing after each sprint
- Manual and automated testing

---

4. Roles and Responsibilities

Role	Responsibilities
Backend Lead	Unit testing for API endpoints, business logic verification, database integrity checks
Frontend Lead	UI testing, form validation, dashboard functionality, API integration testing
QA Lead	Create test cases, perform system and integration testing, report bugs, validate fixes
DevOps Lead	Ensure staging/production environments for testing, monitor performance and logging

---

5. Testing Types

5.1 Unit Testing

- Test individual functions and modules
- Backend: ASP.NET Core API functions, business logic, authentication, application status updates
- Frontend: React components, forms, input validation, state management

5.2 Integration Testing

- Test communication between frontend and backend
- API endpoints for internships, projects, competitions

- User workflows across portals

5.3 System Testing

- Full system testing in deployed Azure environment
- Verify end-to-end scenarios for all user roles
- Ensure all modules work together

5.4 Functional Testing

- Student portal: login, browse internships/projects/competitions, apply, track status
- Company portal: review applications, accept/reject, schedule interviews
- Admin portal: approve companies, manage internships/projects/competitions, generate reports

5.5 Non-Functional Testing

- **Performance:** Load testing with multiple users
- **Security:** Test authentication, authorization, data protection
- **Usability:** User interface intuitiveness, navigation, responsiveness
- **Compatibility:** Browser compatibility (Chrome, Edge, Firefox)
- **Reliability:** Data consistency, fault tolerance, backup verification

5.6 Regression Testing

- After each sprint or feature update
- Ensure new features do not break existing functionality

5.7 Acceptance Testing

- Conducted before final deployment
- Verify system meets all SRS and stakeholder requirements
- User acceptance testing for students, companies, and admin

---

6. Test Case Examples

Test Case ID	Module	Description	Expected Result
TC001	Authentication	Verify student login with valid credentials	Student is logged in and redirected to dashboard
TC002	Internship	Student applies for internship	Application status set to 'Applied', confirmation notification sent
TC003	Company	Company accepts student application	Application status updated to 'Shortlisted', student notified
TC004	Project	Student requests participation in project	Request recorded, status 'Pending', project leader notified
TC005	Competition	Student registers for competition	Registration saved, status 'Registered', confirmation displayed
TC006	Security	Unauthorized access to admin portal	Access denied, proper error message displayed
TC007	Performance	Load test 500 concurrent users	System response < 2 seconds, no crashes

---

7. Test Environment

- **Frontend:** Azure Static Web App / App Service
- **Backend:** Azure App Service
- **Database:** Azure MySQL
- **Storage:** Azure Blob Storage
- **Testing Tools:** Postman, Selenium, Jest, JMeter (for load testing)
- **Browsers:** Chrome, Edge, Firefox

---

8. Defect Reporting and Tracking

- All defects logged in GitHub Issues or Jira

- Severity levels: Critical, Major, Minor
- Assign responsible team members for fixing
- Retest after fixes
- Maintain defect log for project evaluation

---

9. Testing Schedule

Phase	Duration	Activities
Unit Testing	Weeks 5–9	Test backend APIs, frontend components during development
Integration Testing	Weeks 9–11	Test frontend-backend communication and workflows
System Testing	Weeks 12–13	Full system testing on Azure environment
Performance & Security Testing	Week 13	Load, reliability, and security testing
Regression Testing	Ongoing	After each sprint update
User Acceptance Testing	Week 15	Stakeholder validation before final deployment

---

10. Exit Criteria

- All critical and major defects resolved
- System meets SRS specifications
- User acceptance testing successfully completed
- Performance, security, and usability requirements fulfilled